Module 14 — Streamlit Dashboard (Frontend)

app/streamlit_recommendation_dashboard.py • 551 lines

In one sentence

The visual console - built for whoever has to explain a recommendation.

1. What this module does

This is the human-facing side of the system. It is designed for the person who has to answer *'why did the site show that to this customer?'* - a merchandiser, a category manager, or an analyst chasing a complaint.

Five tabs map onto what the marking rubric asks a dashboard to demonstrate: the recommendations themselves with business context, the explanation behind them, content-similar products, a cold-start demonstration, and the customer's history.

Like the API, it holds no scoring logic. It loads the same HybridRecommender.

2. Concepts covered

Every idea this module uses, in plain terms.

Concept	In simple terms	Where it is used here
Streamlit	Turn a Python script into a web app	The framework used
Resource caching	Load heavy objects once per session	@st.cache_resource
Reactive re-execution	The script reruns on every interaction	Why caching is essential
Session state	Remembering what the user selected	Widget selections
Tabs and layout	Organising a dense interface	st.tabs, st.columns
Data visualisation	Charts for category mix and attribution	st.bar_chart
Progressive disclosure	Detail behind expanders	st.expander
Cohort selection	Meaningful groups, not a raw list	The sidebar radio
Honest UI signalling	Colour-code how trustworthy a result is	Green / amber / red strategy label

3. How it works, step by step

1. **Load the recommender once per session** with cache_resource. Streamlit reruns the whole script on every click, so without caching the model would reload each time.
2. **Show cohort choices in the sidebar** - highly active, typical, cold start, or any. Picking a customer at random almost always lands on a light user and makes the personalisation look weaker than it is.
3. **Display the customer's profile** - segment, preferred category, interaction count, total spend.
4. **Show the serving strategy** in colour: green for personalised, amber for cold start, red for unknown.
5. **Render the five tabs**, each answering a different question.

4. Key functions

Element	What it does
load_recommender()	Cached loader for the service object
STRATEGY_LABELS	Maps each strategy to a label and a colour
Sidebar cohort selector	Jump to an interesting customer quickly
Five tab blocks	One per question the operator might have

The five tabs

Tab	What it shows	Rubric criterion
Recommendations	Ranked products, basket value, category mix, signal chart	Business recommendation display
Explainability	Attribution chart, evidence, content justification	Explainability visualisation
Similar Items	Content neighbours, with a warning on cold items	UI usability
Cold-Start Demo	Three scenarios: new customer, unknown, new product	Cold-start handling
Customer History	Purchases, conversion rate, category spread	UI clarity

5. Inputs, outputs and how to run

	Detail
Inputs	All model artifacts
Outputs	An interactive web page
URL	http://localhost:8501
Run with	streamlit run app/streamlit_recommendation_dashboard.py

6. Key code

app/streamlit_recommendation_dashboard.py — load_recommender()

```
51 | def load_recommender():
52 |     """Load the recommender once per session."""
53 |     return HybridRecommender.load()
```

cache_resource, not cache_data - see the note below.

7. Things to remember

Why cache_resource rather than cache_data

This object holds several hundred megabytes of similarity matrices and a PyTorch model, none of which is serialisable in the way cache_data expects. cache_resource keeps the live object in memory instead of trying to pickle it.

The cohort selector is a deliberate demo decision

With 6,000 customers in a dropdown, a random pick almost always lands on somebody with three interactions, and the recommendations look weak. Offering 'highly active' and 'cold start' as explicit choices lets a reviewer jump straight to the case they want to see.

8. Likely questions

Q. Why build a dashboard when you already have an API?

A. They serve different people. The API is for other systems; the dashboard is for a human who needs to understand a decision. The business case allows either - having both means the explainability work can actually be seen rather than read out of a JSON response.

Q. Walk me through the cold-start tab.

A. It has three scenarios. A registered customer with no activity gets recommendations from their declared category and their segment's price band - note the prices stay low for a Budget Shopper. A completely unknown ID falls back to de-biased global popularity. And a brand-new product with zero sales is matched to neighbours purely from its description.

Q. What does the explainability tab show?

A. For any recommended product: the sentence explaining why, a bar chart of which signal contributed what share, the customer's own comparable purchases, how many similar customers rated it well, and the exact words that link it to what they bought before.